

ESE3600 Project Report

Ohm Sweet Ohm

James Steeman and Timothy Zhang

Introduction:

TinyML enables low-power and consistent identification of resistors, addressing the persistent laboratory challenge of organizing components that are visually difficult to distinguish. Because manual verification via multimeters is tedious and misidentification can lead to circuit failure, we developed a standalone system using the ESP32 Xiao Sense to automate classification. This report details our methodology in developing the system for robust real-world performance.

Motivation:

Resistors are one of the most common, yet difficult to organize components. They often end up scattered throughout the lab after use, which leads to a lot of waste, throwing them away, or arguably worse, returning them to the wrong/random bin. Currently, for students who cannot read resistor code, the only way to correctly return resistors is to use a DMM to measure resistance. This could become tedious especially when the numbers of resistors increase. Over years in the Detkin lab, we've seen many students have issues where resistor bins being improperly sorted or mixed lead to their circuits behaving improperly. This often leaves the students confused, and sometimes results in damage to their components and circuit, which leads to additional headache.

Dataset Details:

We collected a custom dataset to ensure that training data would match our inference setting in the most accurate way possible, this would help the final accuracy as all the "learned features" would actually apply to the real use case. The dataset includes 3765 photos across 9 resistor values + 1 idle no resistor state. All resistors chosen were 4 band resistors with a tan casing color, so nothing but the color code would be able to distinguish the resistors. We chose resistors such that color bands would be difficult to distinguish, including several that were 10x multiples of each other and therefore 3 of 4 bands common.

This process involved using the same camera as the one at inference (OV3660 on the xiao) and with the same capture distance and consistent white background as controlled during inference. The resulting image had extremely poor quality but this was exactly the goal as it would make no sense to train on high quality images and have the deployment on poor quality captures. In essence, the model will "learn" to adapt to this poor quality.

The camera library allows for different capture resolutions. The one selected was "FRAME_SIZE_QVGA" as this outputs 320x240, the closest dimension to our expected model input(224x224)

A capture script was written to capture 100 images per run from our system at one second intervals and sent over the data through serial for saving. During that, we moved the camera unit so each image included a different rotation and centering/offset, as well as unique lighting/shadowing. 4 runs were captured for each resistor class with each run having a different resistor leg bend pattern.

All images were manually viewed using icon preview to filter out images that were significantly too blurry, partially out of frame, etc. The first image of every capture cycle was also removed due to a green tinting that we did not further diagnose. This resulted in a final 340~400 usable images per class for the total 3765 image dataset, examples of which can be seen in the appendix figure A1.

Model Choice/Design:

We believed that the resistor classification task could be more complex for vision recognition than something done previously in the course like the MNIST dataset using a few custom defined convolution layers, as the resistor color code contains both spatial and color information, which can be anywhere in the image and in any orientation. From the Assignment 3 rock paper scissor task, we recalled using MobilenetV2 for feature extraction and performing transfer learning to specialize classification for our own task. We took this as our initial approach, taking the MobilenetV2 feature extraction layers and applying our own single dense classification layer based on the number of labels in our dataset. The single dense classification layer added minimal learnable parameters, and therefore helps with model size and inference speed when deployed.

We experimented with several aspects of this model, most particularly the alpha value for MobilenetV2 changing the width of the feature extraction layer. We started with no modification to alpha, but soon moved to alpha taking the value 0.5. We also tested with alpha having value 0.35 (see appendix Figure A2), but for our final 10 class model, we found the 0.5 alpha version to be slightly more reliable and worth the inference time tradeoff, so finished the project demonstrations with our 0.5 alpha 10 class model. We briefly experimented with an additional dense classification layer in some of our models, but this was ultimately unnecessary.

To summarize, our final 10 class model was a MobileNetV2 with alpha equal to 0.5 feature extraction block with a global average pooling layer before a single densely connected 10 output classification layer. A 0.35 dropout layer was included to help prevent overfitting.

Figure 1. Final Demo Model Summary

Layer (type)	Output Shape	Param #
mobilenetv2_0.50_224 (Functional)	(None, 7, 7, 1280)	706,224
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 10)	12,810
Total params: 719,034 (2.74 MB)		
Trainable params: 700,490 (2.67 MB)		
Non-trainable params: 18,544 (72.44 KB)		

Model Training and Evaluation Results:

When training our 4 class baseline model, we started with transfer learning as mentioned in the model choice section. Initially, we left the entirety of the MobileNetV2 layers frozen, and only trained the classification layers. While this was successful in training, validation, and test accuracies around 90% in tensorflow, after quantization, the accuracy would drop drastically (~77%). This is due to the disconnect between the frozen layers and newly transferred trained classifications layers. Unfreezing the last several MobileNetV2 layers and achieved greater accuracy in models, but ran into an accuracy bottleneck where training and test accuracy still would not get much higher than 90%.

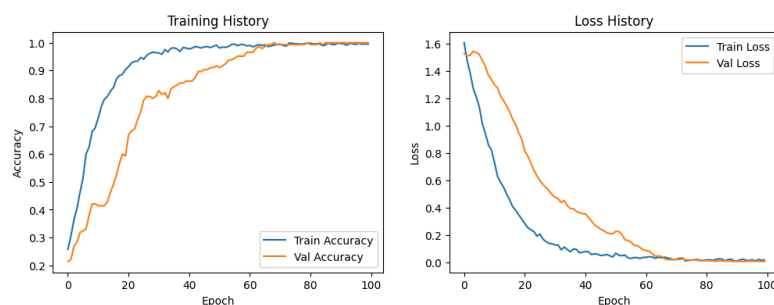
After considering this, we rethought the transfer learning approach. MobilenetV2 was trained on the Imagenet 1k class dataset, which contained clear and in-focus images. Our dataset, on the other hand, contained very specific and likely uncommon shapes for MobileNet. In addition, are dataset intentionally contained out of focus “blurry” images to match expected inference conditions. We also realized that though our dataset was relatively small, especially with only 4 classes at the time, we could consider it to be “large” as it captured much of the expected variation at inference time (rotations, shadows, centering, lead bending). Therefore, as highlighted in our annotated version of the course’s table on transfer learning, Figure 2, training from scratch could yield better accuracy.

Figure 2. Transfer Learning Table

	New dataset is similar to original dataset	New dataset is (less) similar to original dataset
New dataset is small	Best case scenario for transfer learning	Not the best scenario for transfer learning
New dataset is large	Transfer learning will work	Training from scratch might yield better accuracy

We decided to train with the alpha 0.5 MobilenetV2 all layers unfrozen, just taking their architecture for the feature extraction layers and retraining entirely. At this point, we also decreased the initial learning rate significantly. Training still completed in under an hour, and this decision let us reach 100% on validation, and test for the baseline 4 class model. It also retained 100% accuracy after quantization to int8 in TFlite, and performed perfectly when deployed on hardware.

Figure 3. Four Class Model Train and Test Results

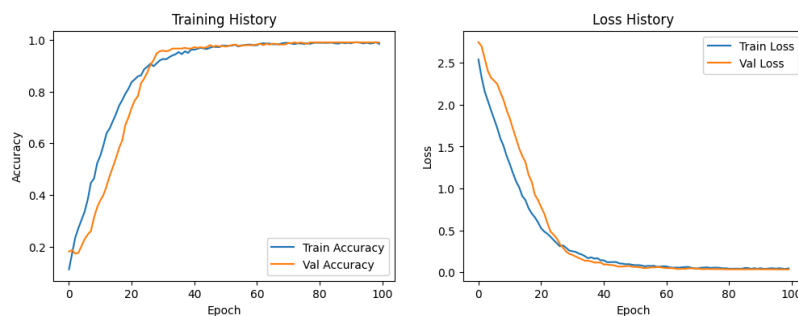


TFLite Model Accuracy: 100.00% (145/145)

We followed the same fully unfrozen approach with our updated 10 class version of the model after collecting additional data to expand the dataset. We had already coded in a dynamic learning rate reduction, which we noticed became very useful as the following models were trained.

Noticing that inference took around a 1.2 with our initial 4 class model, we decided to first attempt a 10 class model with reduced 0.35 alpha for the MobileNetV2. Training again completed in under one hour. This model achieved 98.95% validation accuracy and 99.74% test accuracy, but dropped to 98.69% on the same test dataset after int8 quantization. The key advantage for this model was the 0.9s inference, about 0.3s faster than the alpha 0.5 versions; however, its performance was a bit too inconsistent when deployed on hardware. 9 classes operated nearly flawlessly, but the 4.7MΩ class having generally low confidence led to frequent false predictions.

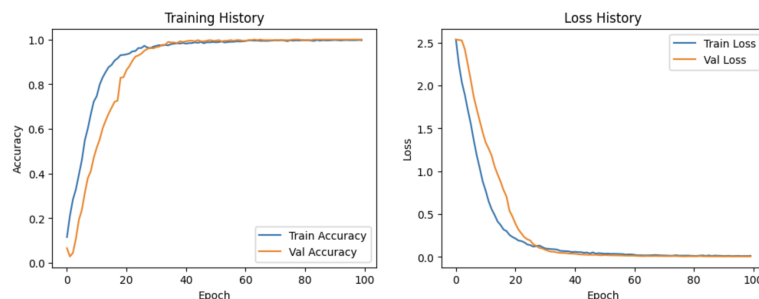
Figure 4. Ten Class Alpha 0.35 Model Train and Test Results



TFLite Model Accuracy: 98.69% (376/381)

We then restored alpha to 0.5 and trained another 10 class model in the same manner. It achieved 100% accuracy in validation and 99.74% for testing before and after int8 quantization. This model was much more consistent and did not make frequent noticeable mistakes. Though the accuracies were similar, the classification confidence was usually much higher in all classes. This alleviated the issue with the previously problematic 4.7MΩ class, however had a slower inference of approximately 1.2s. We chose this model to use for our final demonstrations as we preferred better confidence and accuracy to minimally faster inference.

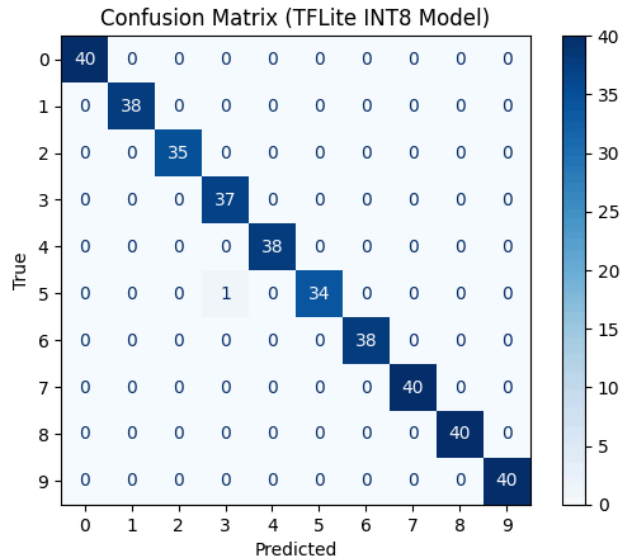
Figure 5. Ten Class Alpha 0.5 Model Train and Test Results



TFLite Model Accuracy: 99.74% (380/381)

The resultant confusion matrix on test data after quantization is shown below.

Figure 6. Final Ten Class Model Confusion Matrix



Deployment/Hardware Details:

See Figure A3 in the appendix for images of our system hardware.

ESP32 Xiao Sense:

This is the heart of all the unit with the on-board camera (OV3660) used for capturing images of resistors, and the ESP32-S3 microcontroller handling the preprocessing (rescaling), model inference, and communicating to the display.

The model was deployed on the ESP32-S3 MCU using TFLite micro. In the code the model was held in a model.cc file, which was generated in our collab notebook.

All the application code was written in VSCode and compiled through platformIO, where it handled sending the camera output to our TFLite micro model.

Display:

We used an Adafruit ST7735R SPI used to output the detection result in an easy and clear interface to the user. This was used instead of the kit display as that display is oriented downwards in our unit's physical configuration. Therefore we placed this external display on top of the unit facing upwards.

We wired the display over an SPI interface with some additional pins as required by the datasheet. The arduino library for the display was added via platformIO.

Mechanical:

Standoffs were used to place cameras at a tuned consistent height that balanced close distance for resolution but also far enough for decent focus. Too far and the image is too small to see the tiny details of different resistor bands and too close the camera focus would be too off and blur the bands together.

Challenges/Future Work:

The main challenge came from realizing and working within our hardware limitations. We had initially proposed a different type of model that would use bounding boxes to extract the color band information from any arbitrary resistor, 4 band or 5 band, and feed that information to a post processing function that knew the resistor rules so that any value resistor could be classified without the model needing to be specifically trained on that value as a class. We found that the camera on the course hardware would not allow us to capture sufficiently clear and detailed images for such analysis. Moreover this type of object detection model would be even slower on our processor, which prompted us to move to the slightly simpler brute force dataset to class model type. From this, we also first tried to use significant data augmentation to train from a minimal dataset, but realized that capturing a substantially larger custom data would be much better for enabling model training for maximal accuracy and robustness.

In order to update to the initially proposed bounding box model, we would likely need to move to a different hardware platform, particularly to have a better camera sensor that would let us use a higher resolution with usable focus. Outside of this, the hardware for the current system is a fairly rapid prototype, which could be improved with additional mechanical design to hide the electronics in a more product-like design.

From a display perspective, we could opt to feed the live capture to the display with an annotated resistance value when detected, this would make it a bit more intuitive for users.

From a model perspective, one improvement would be to replace the MobileNetV2 feature extraction layers with custom feature extraction layers designed to retain the accuracy while having flexibility to reduce trainable parameters and improve inference speed. Since we are no longer using transfer learning and training the entire model from scratch anyway, we need not be limited by the MobileNetV2 architecture constraints. Another possible improvement would be adding a saturation stretch to the image preprocessing in addition to normalization. This may help make the color band information more distinguishable and therefore help improve performance with even the blurrier images from the dataset.

While working on the report, we retrained our 10 class model with a larger learning rate, which resulted in a 13 minute train time with early stopping (~3x speedup). The resulting accuracies were as good or better than our deployed models, however, these were not actually deployed and tested on hardware. The results are shown in the appendix in figures A4 and A5. As we spend little time on tuning hyperparameters after finding sufficient ones to train for high accuracy earlier in the project, this suggests that further work could be performed on hyperparameter tuning to reduce training time while continuing to maximize accuracy.

Appendix:

Colab notebook: [mobilenet_50_10class_3600_resistors.ipynb](#)

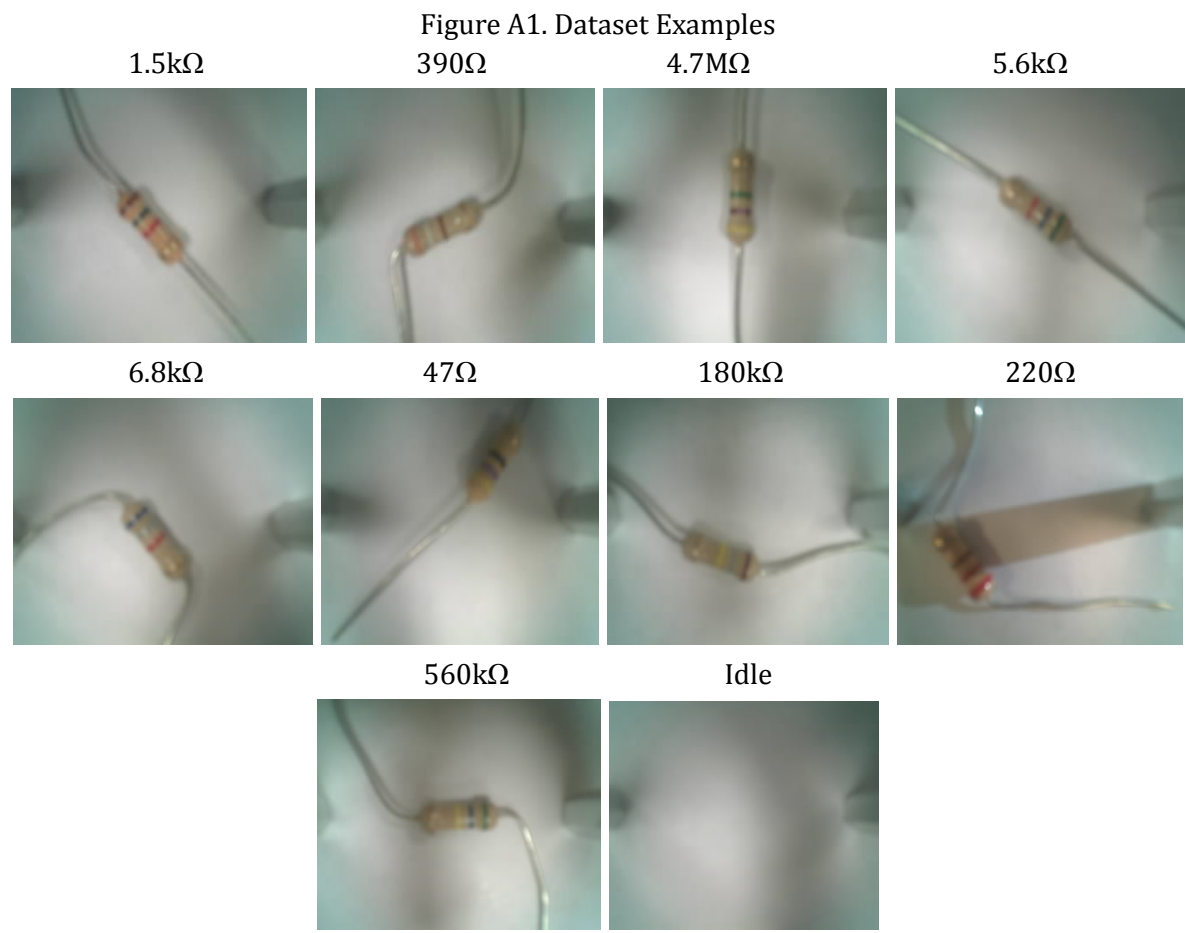


Figure A2. Ten Class Alpha 0.35 Model Summary

Layer (type)	Output Shape	Param #
mobilenetv2_0.35_224 (Functional)	(None, 7, 7, 1280)	410,208
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 1280)	0
dropout_3 (Dropout)	(None, 1280)	0
dense_3 (Dense)	(None, 10)	12,810
Total params: 423,018 (1.61 MB)		
Trainable params: 408,938 (1.56 MB)		
Non-trainable params: 14,080 (55.00 KB)		

Figure A3. Hardware Images

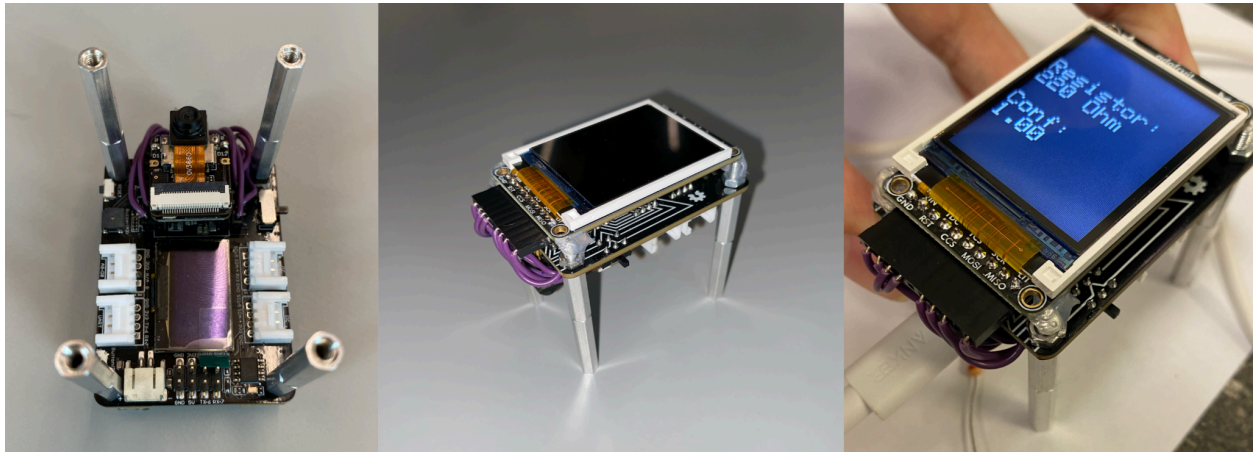


Figure A4. Retrained Ten class model with 0.5 alpha

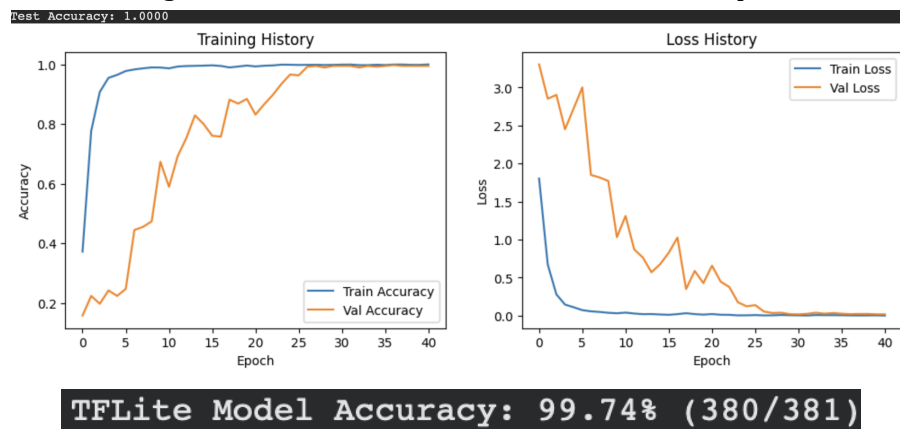


Figure A5. Retrained Ten class model with 0.35 alpha

